Software Architecture and Design Specification

Railway Ticket Reservation System

Version 2.0 | Revised 12 November 2026 | Team 15 | UE24CS341A

This revision specifies the delivered C++ application and its verification evidence. It aligns the submission documents with the implementation and identifies the remaining team acceptance actions. It is prepared for review; signatures and acceptance are not implied.

Team member	USN	Role
Deeptanshu Kumar	PES1UG24AM357	Developer
Amar	PES1UG24AM340	Tester
Aaryaprasad	PES1UG24AM331	Product Owner

Revision history

Version	Date	Change
1.0	30 September 2026	Original submitted baseline; retained in repository history
2.0	12 November 2026	Reconcile actual stack, interfaces, policies, traceability and current verification evidence

Application baseline: 6b5464298c949806ebba0b7bd1d903c962b5255d. The final feature integration is main commit fb1b4d55957ab8022e136396d656bec870c08375, merged by Amar in PR 12 on 3 November 2026. Current evidence was generated on 12 November 2026.

1 Introduction

This design translates the identified requirements into the delivered C++ components, interfaces, database and transaction behavior. It supports maintainability, testability and explanation during the final demo. The original version described React/Express and a generic SQL database; this revision documents the actual implementation and records deviations without inventing approval.

- Course Mini Project Deliverables Parts 1, 2 and 3; evaluation checklist; final demo and viva FAQs.
- Repository README, CMakeLists.txt, src/api, src/services, src/db and tests at the application baseline above.
- IEEE 830 style requirements structure; IEEE 1016 style software design description; IEEE 829 style test planning structure, adapted to the supplied course templates.
- docs/SUBMISSION_INDEX.md for the deliverable inventory; docs/DEVIATIONS.md for baseline differences; docs/reports for actual results.

2 Design context and decisions

Concern	Decision and rationale
Academic C/C++ requirement	Implement all reservation business services and HTTP backend in C++17
Simple browser delivery	Serve static HTML/CSS/JavaScript from the same server; no Node/React runtime needed
Persistent consistent records	Embedded SQLite with foreign keys, prepared statements, transaction mutex, busy timeout and immediate transactions
Testability	Keep routes separate from main; services accept database and test configuration; GoogleTest uses isolated fixtures
Deployment simplicity	Single executable with web assets; Docker image publication through GitHub Actions; hosted service deployment not claimed

3 Architecture

3.1 Pattern and boundaries

Use a layered architecture: presentation, HTTP/API coordination, application services, and persistence. Cross-cutting validation, hashing, authorization, TLS and logging support the services. One server process shares a serialized Database wrapper; tests do not establish independent multi-process scalability.

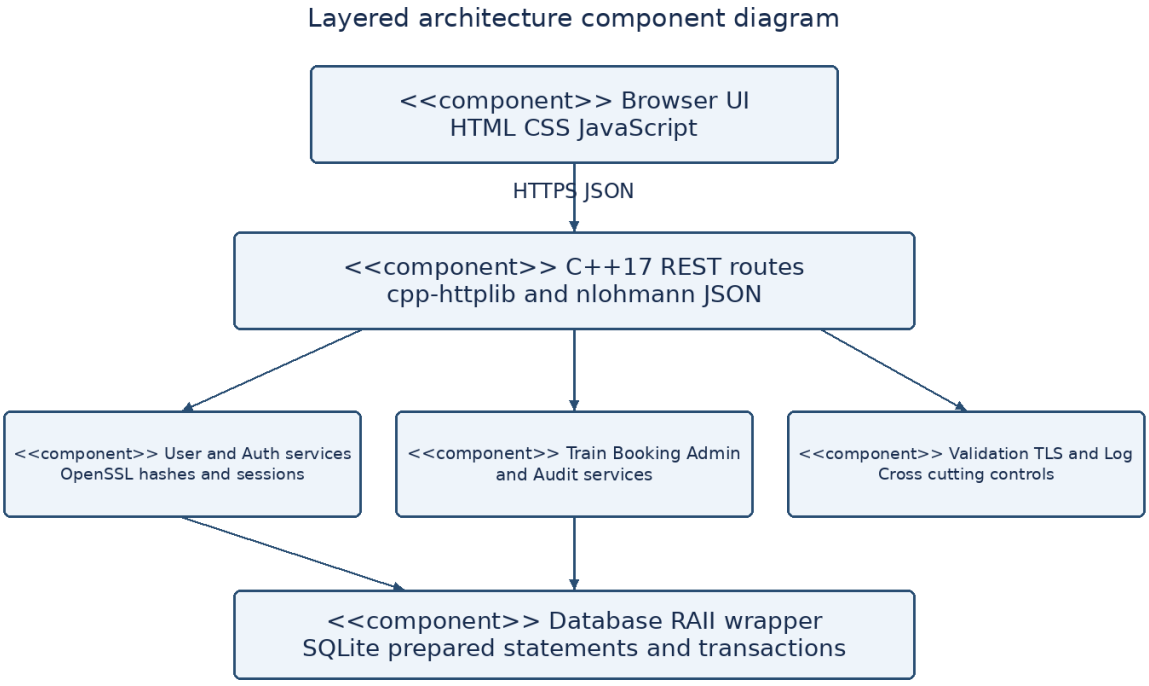


Figure 1 C++ layered component architecture

3.2 Component responsibilities

Component	Responsibility	Requirements
web/	Forms, search/results, train details, preview, ticket/history/cancellation, admin editor	FR001–017;NF007
Routes and AdminRoutes	HTTP/JSON parsing, status codes, authenticated owner/admin gates	All API requirements
UserService and AuthService	Profiles, passwords, opaque sessions, expiry, lockout and revocation	FR001/002; NF005/006; SR001/002
TrainService	Search filters, ordered stops, fares and train/date/class availability	FR003–005
BookingService	Validation, preview, atomic confirmation/PNR/seats, owned cancellation/refund	FR006–016;NF002
AdminTrainService	Transactional add/update/deactivate with reservation safeguards	FR017
AuditService and Log	Booking/cancellation audit and redacted access logging	FR018;NF008;SR005
Database and Schema	RAII connection/statements, mutex, prepared binding and schema constraints	Persistence and consistency
Validation and TlsConfig	Type/length/enum/date checking; certificate/key configuration	NF003;SR003/004

3.3 Technology and data stores

C++17, CMake, cpp-httplib 0.18.3, nlohmann/json 3.11.3, SQLite 3.46.1 amalgamation, OpenSSL and GoogleTest 1.15.2. GCC/gcov and gcovr supply coverage; cppcheck and CodeQL provide static checks. No JWT parser, React build, Express server or MySQL server is needed.

3.4 Security architecture

- PBKDF2-HMAC-SHA256 with random 16-byte salts and default 600000 iterations; configured production minimum 100000. Constant-time comparison and upgrade of older hashes on successful login.
- Random 256-bit opaque bearer token returned once; only SHA-256 token hash stored with 24-hour expiry. Logout deletes session. Password change revokes other sessions. Browser stores token in sessionStorage; no HttpOnly-cookie design is claimed.
- Authorization always derives user from current session. Admin endpoints reread the current database role. Other-owner booking requests return 404 to avoid disclosing existence; passenger admin attempts return 403.
- Bound SQL values and explicit field validation; 64 KB request limit; output escaping in browser code. Server failures return generic JSON, never SQL/stack/credential details.
- TLS certificate/key variables enable an SSLServer. HTTP default is development-only or behind a configured TLS proxy. HSTS depends on HTTPS indication; forwarded headers require a trusted deployment boundary.
- Audit events share booking/cancellation transactions. Application behavior is append-only, not tamper-proof against direct database administrators. Access logs omit query values, bodies, headers and redact sensitive text.

3.5 Data model

Table	Key relationships and stored data
users	id PK; unique email; name, phone, password_hash, role, created_at
sessions	token_hash PK; user_id FK; expires_at
trains	id PK; unique number; name, active flag
train_stops	train_id FK; station, arrival/departure, day_offset, ordered sequence
train_classes	train_id FK; class code/name, fare, total_seats
bookings	id PK; unique PNR; user/train/class/date; status, stored fare, quantity, cancel/refund fields
passengers	booking_id FK; name, age, gender, assigned seat
seat_reservations	booking/train/date/class; reserved quantity used by availability
audit_events	event type, time, actor, booking, train, PNR and refund identifiers

There is no separate payment or cancellation table. Fare is snapshotted in bookings; current train/class labels are loaded when displaying details. A duplicate-seat trigger rejects conflicting confirmed seat allocations. Cancellation removes inventory reservations while preserving ticket/passenger history. Admin updates cannot remove referenced classes or reduce capacity below existing reservations/assigned seat numbers.

4 Detailed design

4.1 Booking algorithm

Validate the authenticated user, train/date/class and one to six passengers (name, age 1–120, gender M/F/O). Preview calculates current fare/quantity and holds no inventory. Confirmation enters BEGIN IMMEDIATE, rechecks active train/class/availability, chooses free seat numbers, generates a random ten-digit unique PNR, inserts booking/passengers/reservation and audit, then commits. Exceptions roll back all writes. Displayed availability is provisional; no idempotency protection is provided for successful client retries.

4.2 Booking sequence

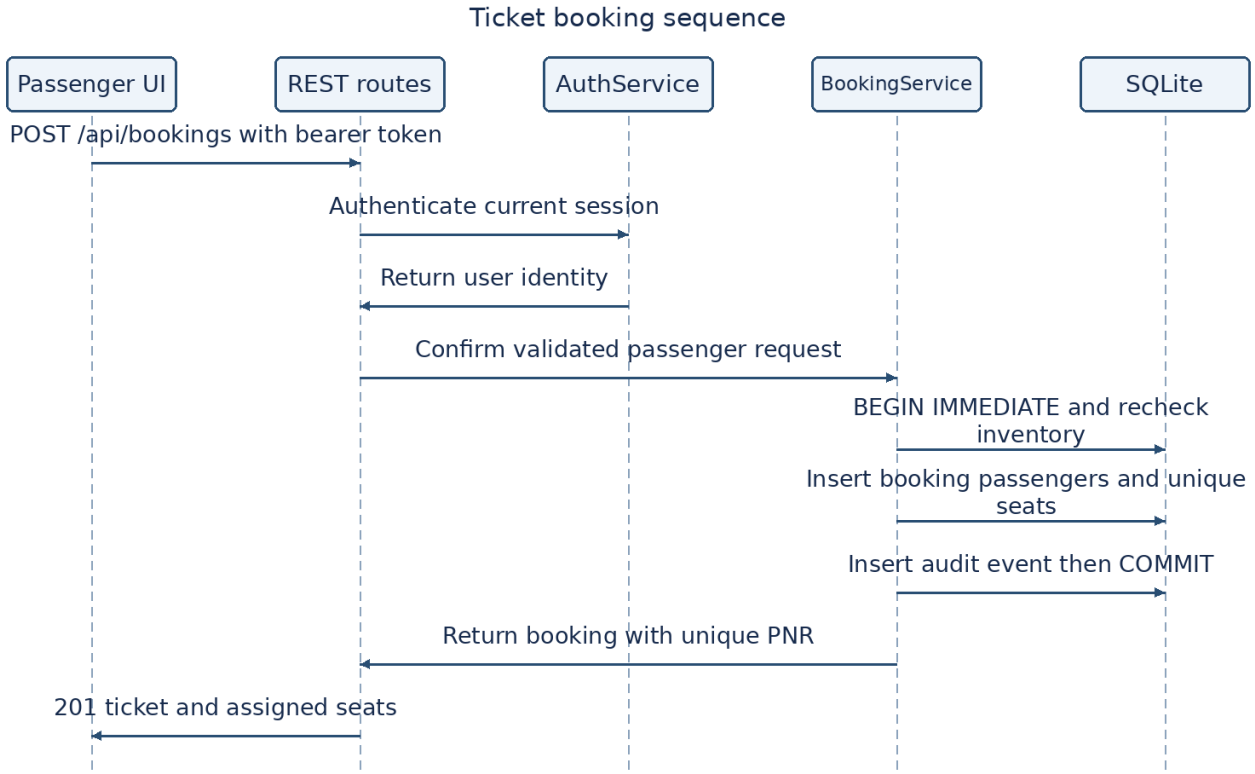


Figure 2 Successful booking sequence

Seat and audit changes occur in one transaction. Validation, authorization and conflict alternatives return before a successful confirmation; rollback removes partial writes.

4.3 Cancellation sequence

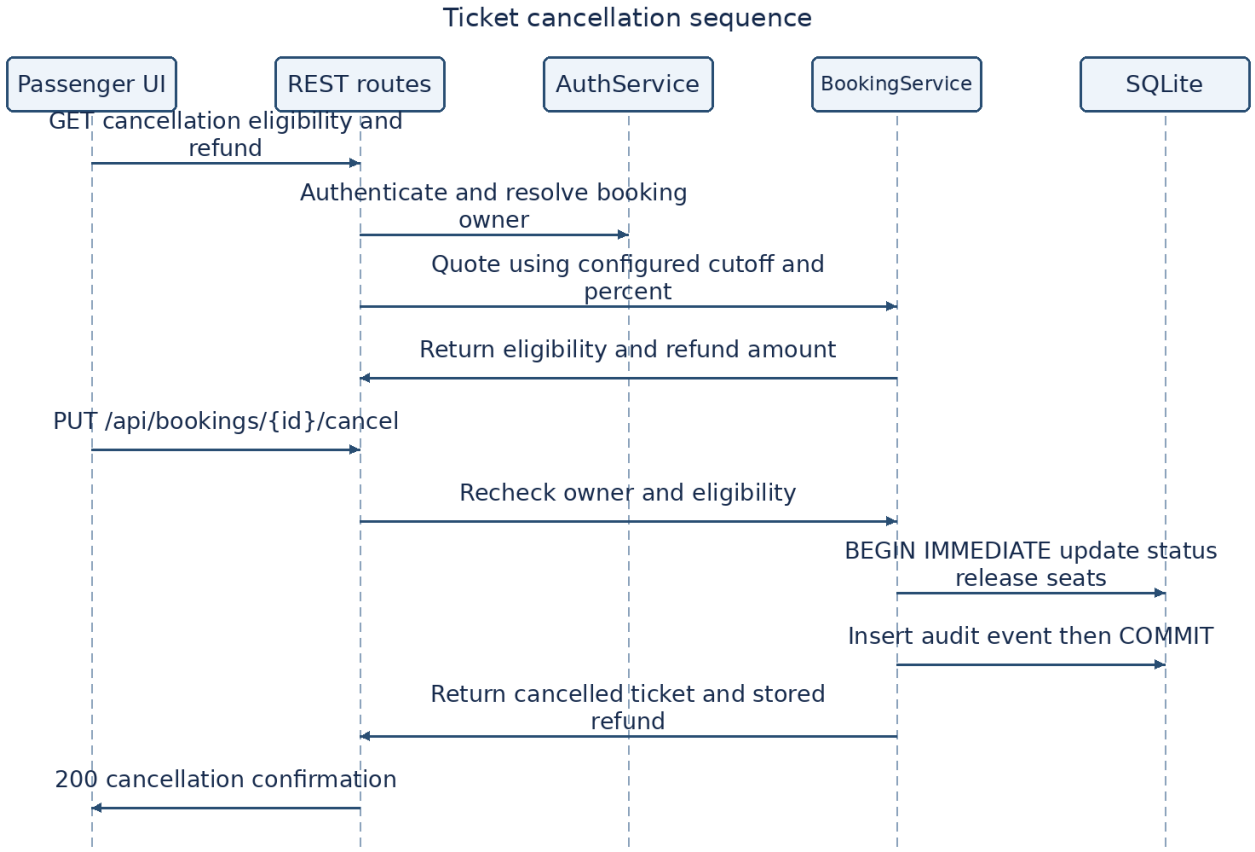


Figure 3 Owner cancellation with refund recording

Quote and confirmation both enforce ownership. Confirmation rechecks eligibility using the current first-stop departure in UTC, marks the whole booking CANCELLED, records timestamp and floor(totalFare × percent / 100), removes reservation rows and records audit atomically. Repeated cancellation returns conflict without another refund/restoration.

4.4 API specification

Method and endpoint	Inputs	Result	Access
POST /api/auth/register	name,email,phone,password	201 user; 400 fields; 409 duplicate	Public
POST /api/auth/login	email,password	200 token, expiry, user; 401/429	Public
POST /api/auth/logout	Bearer header	200 ended session;401	Session
GET /api/users/me	Bearer header	200 own user	Session
PUT /api/users/me	name,phone	200 own profile; 400 fields	Session
PUT /api/users/me/password	currentPassword, newPassword	200;revoke other sessions	Session
GET /api/trains/search	source, destination, date; optional class, maxFare, departureFrom/To, sort, order	200 trains/date; 400 fields	Public
GET /api/trains/{id}/schedule	Numeric train ID	200 ordered stops;404	Public active train
GET /api/trains/{id}/availability	date query	200 classes with counts; 400/404	Public active train
GET /api/trains/{id}/fares	Numeric train ID	200 class fares;404	Public active train
POST /api/bookings/preview	trainId, date, class, passengers	200 preview;400/409	Session
POST /api/bookings	Same booking fields	201 booking and PNR; 400/409	Session
GET /api/bookings	Bearer header	200 own bookings	Session

Method and endpoint	Inputs	Result	Access
GET /api/bookings/{pnr}	Numeric PNR, not internal ID	200 ticket; 404 other owner/unknown	Owner
GET /api/bookings/{id}/cancellation	Internal booking ID	200 eligibility/refund quote; 404	Owner
PUT or POST /api/bookings/{id}/cancel	Internal booking ID	200 cancelled ticket; 409 ineligible	Owner
GET /api/admin/me	Bearer header	200 admin;401/403	Admin
GET /api/admin/trains	Bearer header	200 full train records	Admin
GET /api/admin/trains/{id}	Numeric train ID	200 full record;404	Admin
POST /api/admin/trains	number,name,stops,classes	201; 400 invalid; 409 duplicate	Admin
PUT /api/admin/trains/{id}	Replacement record	200; 409 capacity/class conflict	Admin
PUT /api/admin/trains/{id}/deactivate	Numeric train ID	200 inactive service	Admin
GET /api/admin/audit	limit 1–500, default 100	200 newest events first	Admin
GET /api/health	None	200 status ok	Public

4.5 Payload contracts

Booking example: {"trainId":2, "date":"YYYY-MM-DD", "class":"SL", "passengers":[{"name":"Demo Passenger", "age":"22", "gender":"M"}]}. The authenticated user ID is never accepted from the body. Registration requires phone as well as name, email, password. Dates use the key date, not the original SAD placeholder journeyDate. Ticket retrieval uses PNR; cancellation uses internal numeric booking ID.

Train payloads contain five-digit number, name, ordered stops and classes. Stops have station, arrival, departure and integer dayOffset 0–30; origin arrival and terminal departure are blank. Classes have code SL/3A/2A/1A/CC/EC, name, integer fare 1–1000000 and totalSeats 1–10000. Updates replace the submitted record atomically, subject to booking safeguards.

4.6 Error handling and observability

Status	Meaning	State invariant
400	Malformed JSON, wrong types, invalid fields/date	No booking/admin mutation on validation failure
401	Absent, expired, revoked or invalid session	Protected resources unavailable
403	Authenticated passenger accessing admin operation	No admin write
404	Unknown or other-owner booking; inactive public train	No resource-existence disclosure for private booking
409	Sold out, duplicate email/train, ineligible cancel, capacity conflict	No partial write or duplicate restoration
413 /405	Oversized payload/unsupported method	Generic JSON failure
429	Login attempt lockout	No successful session issued while locked
500	Unexpected internal failure	Generic response; transaction rollback

5 Deployment and operations

Build with cmake -S . -B build and cmake --build build -j. Run rtrs_server with RTRS_DB pointing to a writable SQLite file and RTRS_WEB_DIR to web assets. RTRS_PORT defaults to 8080. Set both RTRS_TLS_CERT and RTRS_TLS_KEY for direct HTTPS. The local --promote-admin EMAIL command grants a registered account the role and revokes its existing sessions. There are no default admin credentials.

CI builds/tests, stores raw XML/coverage, runs actual executable validation, cppcheck, CodeQL and a Docker smoke test. Main-branch CD publishes an image to GHCR; it does not deploy a hosted service.

Backup/restore must account for SQLite WAL, and a clean same-database restart preserves committed records. Current system evidence covers selected restart/forced-stop scenarios, not all storage failures.

6 Traceability and validation

Requirement	Module	Test case references
RTS-F-001	UserService / Routes	TC-REG-01, TC-REG-02
RTS-F-002	AuthService / Routes	TC-AUTH-01, TC-AUTH-02, TC-AUTH-03, TC-AUTH-04
RTS-F-003	TrainService / Routes	TC-SEARCH-01, TC-SEARCH-03, TC-SEARCH-04
RTS-F-004	TrainService / web/app.js	TC-SEARCH-02
RTS-F-005	TrainService / seat_reservations	TC-AVAIL-01, TC-AVAIL-02
RTS-F-006	BookingService / web/app.js	TC-BOOK-01
RTS-F-007	BookingService / Validation	TC-BOOK-02, TC-BOOK-06
RTS-F-008	BookingService / Database	TC-BOOK-03
RTS-F-009	BookingService / bookings	TC-BOOK-04
RTS-F-010	BookingService / Database	TC-BOOK-05, TC-CONC-01
RTS-F-011	BookingService / web/app.js	TC-TICKET-01
RTS-F-012	BookingService / Routes	TC-HISTORY-01
RTS-F-013	BookingService	TC-CANCEL-01, TC-CANCEL-04
RTS-F-014	BookingService	TC-CANCEL-02
RTS-F-015	BookingService / Database	TC-CANCEL-03, TC-CANCEL-05
RTS-F-016	Authorization / BookingService	TC-SEC-03
RTS-F-017	AdminTrainService / AdminRoutes	TC-ADMIN-01, TC-ADMIN-02, TC-ADMIN-03, TC-ADMIN-04
RTS-F-018	AuditService / audit_events	TC-AUDIT-01, TC-AUDIT-03
RTS-NF-001	TrainService / BookingService / browser	TC-PERF-01
RTS-NF-002	Database / BookingService	TC-CONC-01, TC-CONC-02
RTS-NF-003	Routes / Validation	TC-ERR-01, TC-ERR-02
RTS-NF-004	Database / executable	TC-REC-01, TC-CANCEL-05
RTS-NF-005	PasswordHasher / UserService	TC-SEC-01
RTS-NF-006	Authorization / AuthService	TC-SEC-02, TC-ADMIN-02
RTS-NF-007	web/index.html / style.css / app.js	TC-UX-01
RTS-NF-008	AuditService / Database	TC-AUDIT-01, TC-AUDIT-03, TC-REC-01
RTS-SR-001	PasswordHasher / AuthService	TC-SEC-01, TC-SEC-07
RTS-SR-002	AuthService / Authorization	TC-AUTH-03, TC-SEC-02, TC-SEC-03
RTS-SR-003	TlsConfig / main / cpp-httplib	TC-SEC-04
RTS-SR-004	Validation / Database / web/app.js	TC-SEC-05, TC-BOOK-02, TC-ERR-02
RTS-SR-005	Log / Routes / web/app.js	TC-SEC-06, TC-AUDIT-02

Named automated tests are listed in the Test Plan and source/test_cases.json. Passing unit/API checks, system assertions and selected browser observations are distinct evidence types. Human UAT and PO/mentor acceptance remain pending; the matrix is not a fabricated sign-off.

Refer to the four reports and browser observations in docs/reports. Local results are 201 passing tests, 122 passing system assertions, 94.7% line coverage and 46.7% branch coverage in the stated scope. A trusted temporary test certificate established a real TLS 1.3 handshake. Human review and policy acceptance remain pending.

7 Limitations and future work

- Add operating-day/future-date rules, segment inventory/pricing and richer journey snapshots if future requirements demand them.
- Introduce request idempotency before treating network retries as exactly-once reservations.
- Evaluate multiple independent SQLite connections/processes and larger realistic load before scale claims.
- Review policy defaults, current-schedule cancellation effects, browser token storage and production proxy trust.
- No real payment settlement, external railway integration or tamper-proof audit store is claimed.

Reviewer	Review responsibility	Signature and actual date
Amar	Test Plan and verification review	Pending
Aaryaprasad	Requirements and policy acceptance	Pending
Rajeshwari Adarakatti	Mentor evaluation	Pending